

Reconstructing electron kinematics from calorimeter showers using machine learning

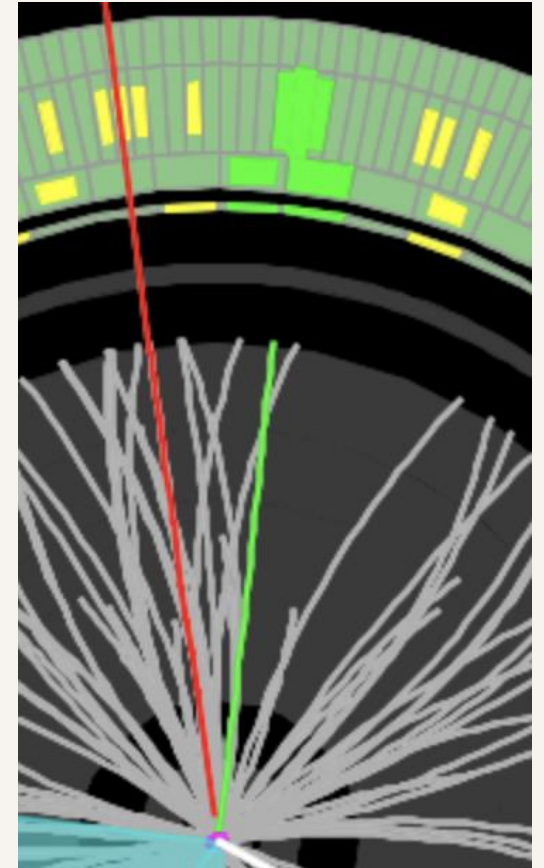
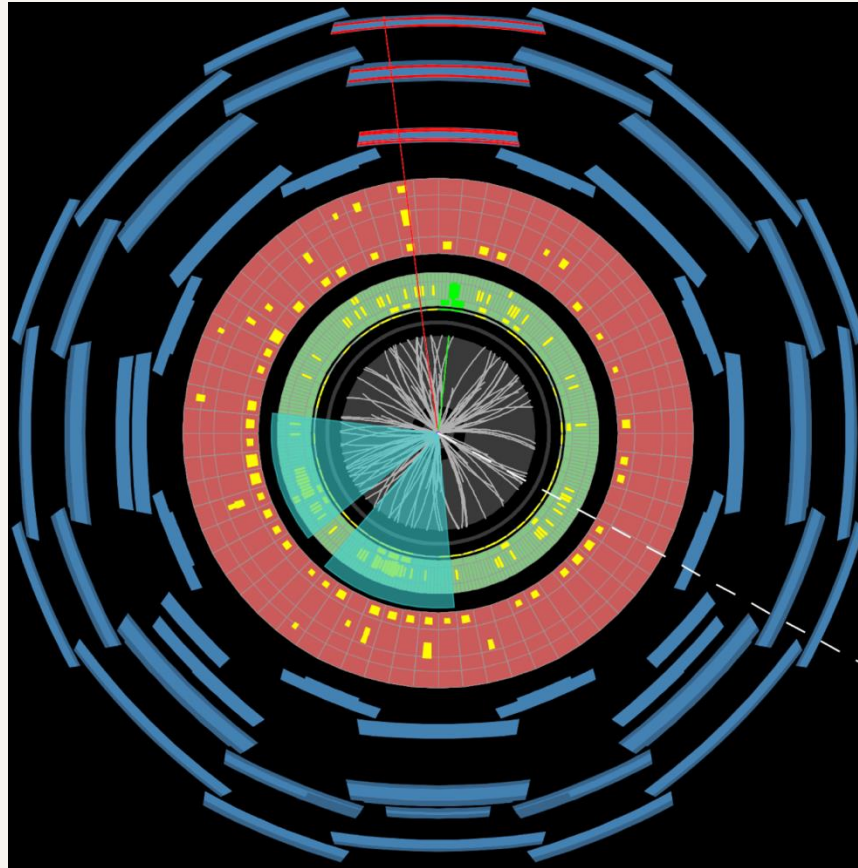
Using the CERN ColliderML Release 1 dataset

Loïc Lambert · Minh Tuan Pham  Corentin Allaire · June 29, 2026

Calorimeter showers

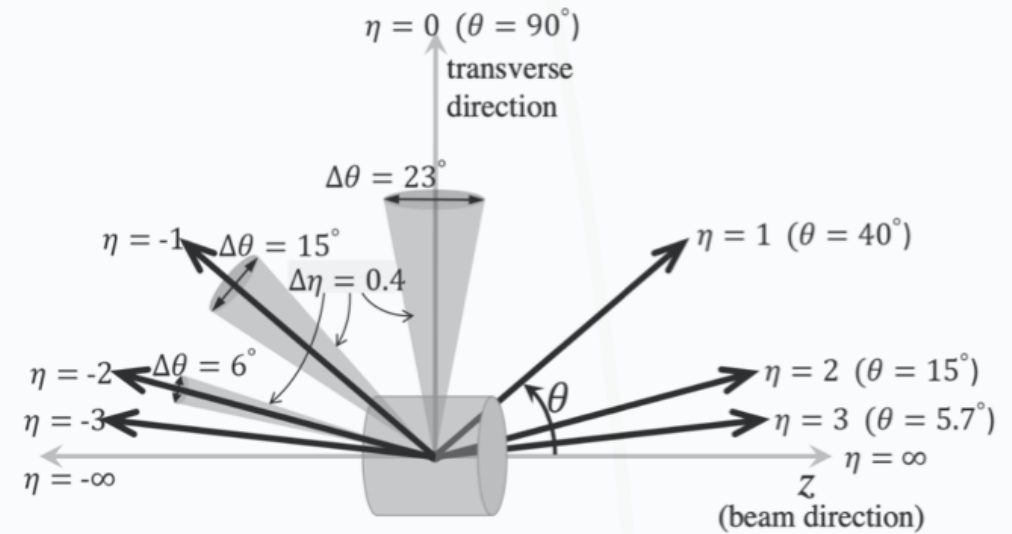
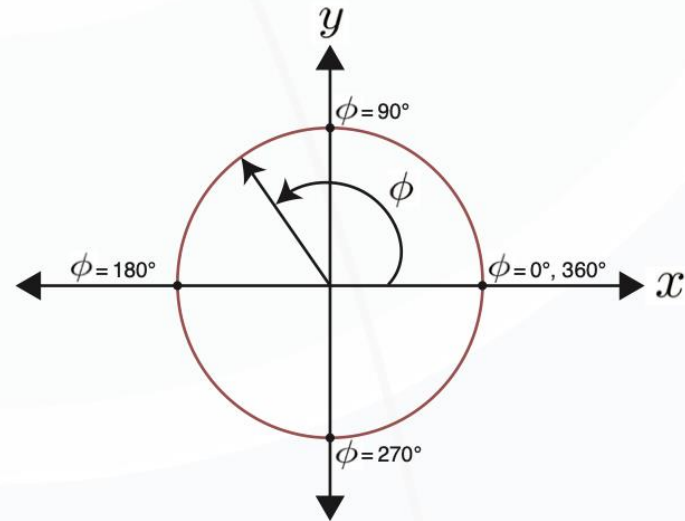
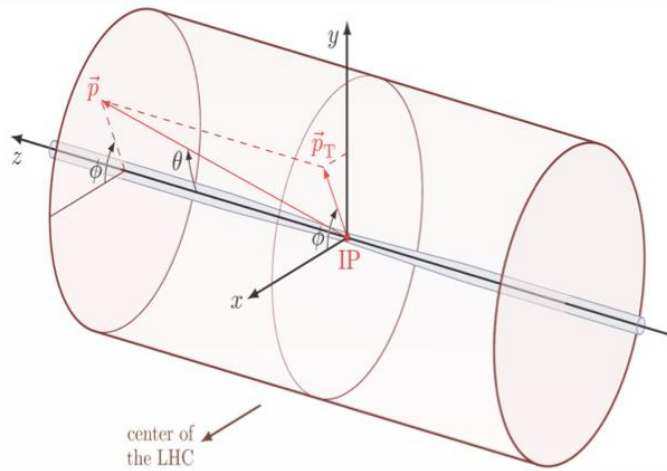
An electron triggers a shower, releasing particles via a chain reaction.

The shower is read as energy spread out over many calorimeter cells.



Electron kinematics

Predicted parameters: (η, ϕ, p_T, z_0)



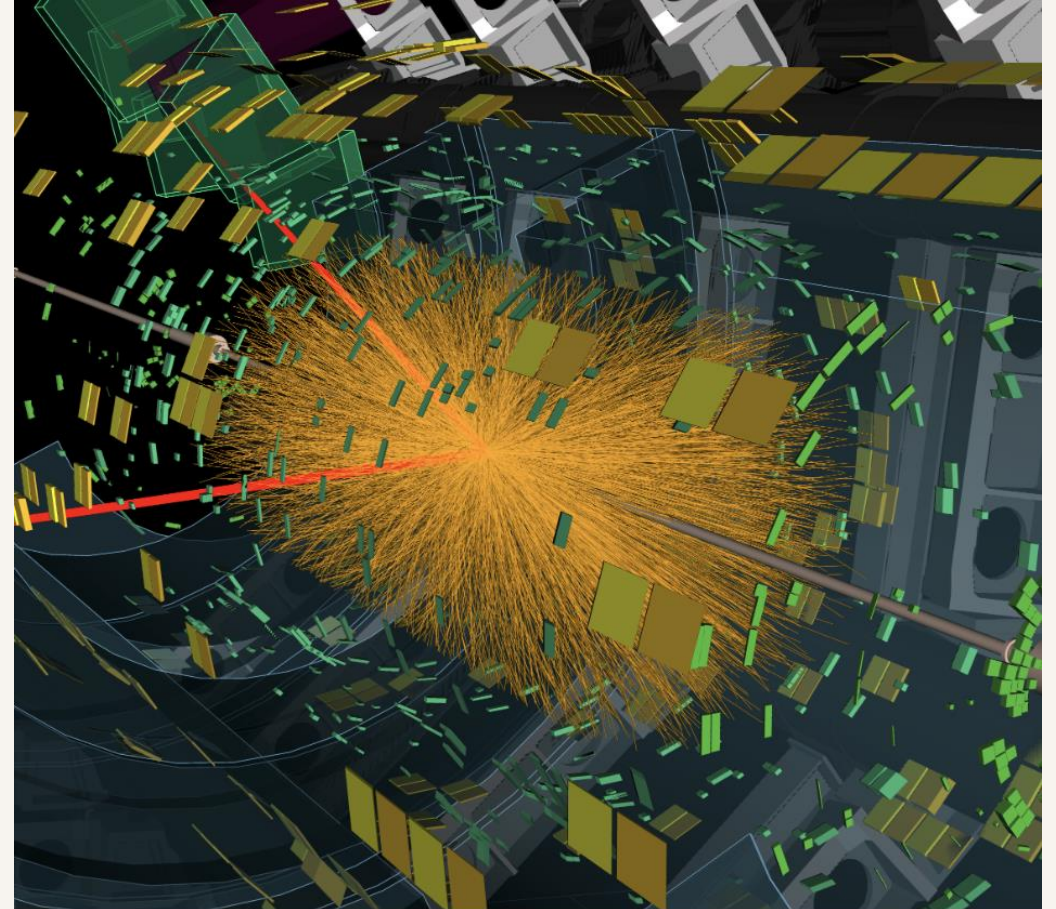
The reconstruction problem

Input: calorimeter cells for one electron

Target: (η, ϕ, p_T, z_0)

Difficulty

- Signal vs. Pileup
- Track parameters from calorimeter information

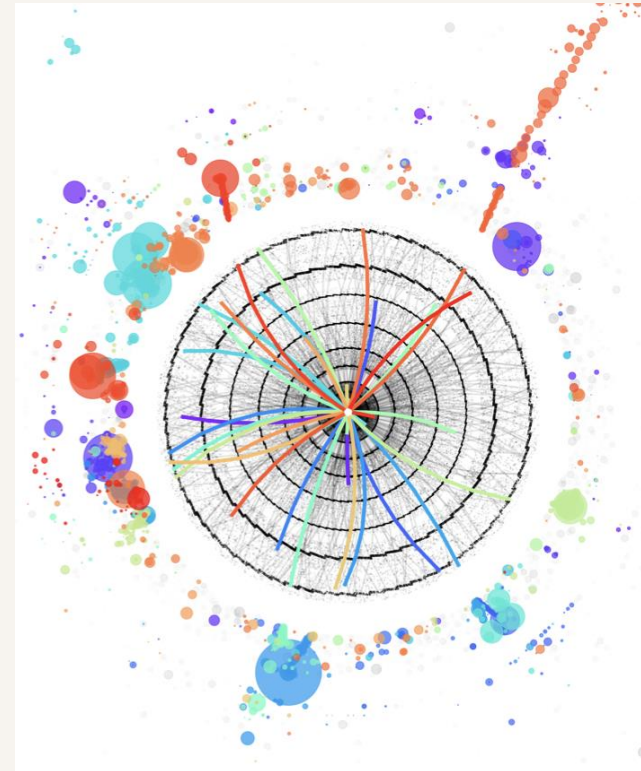
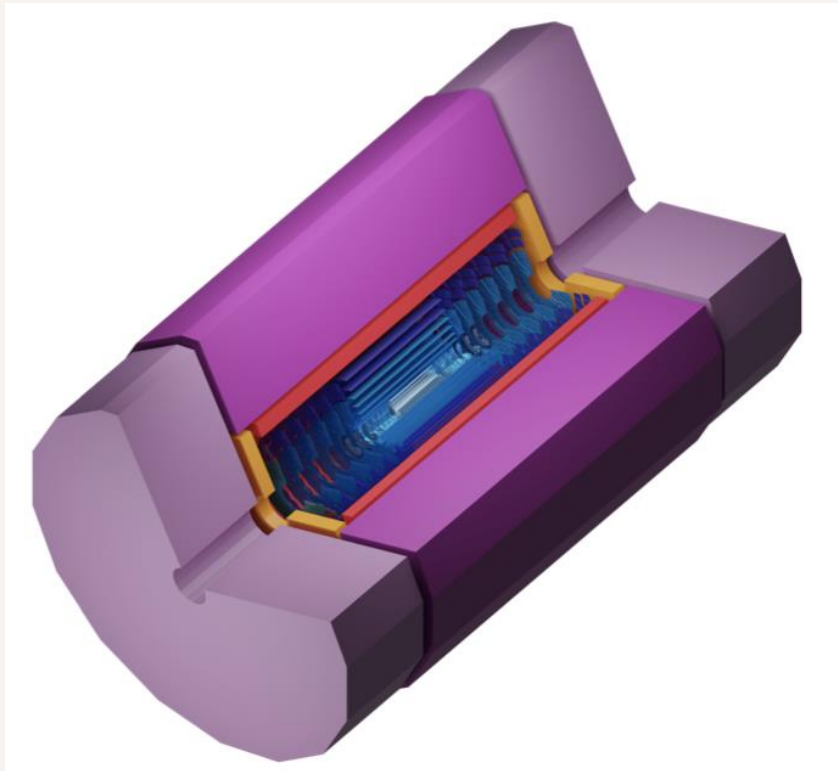


Method

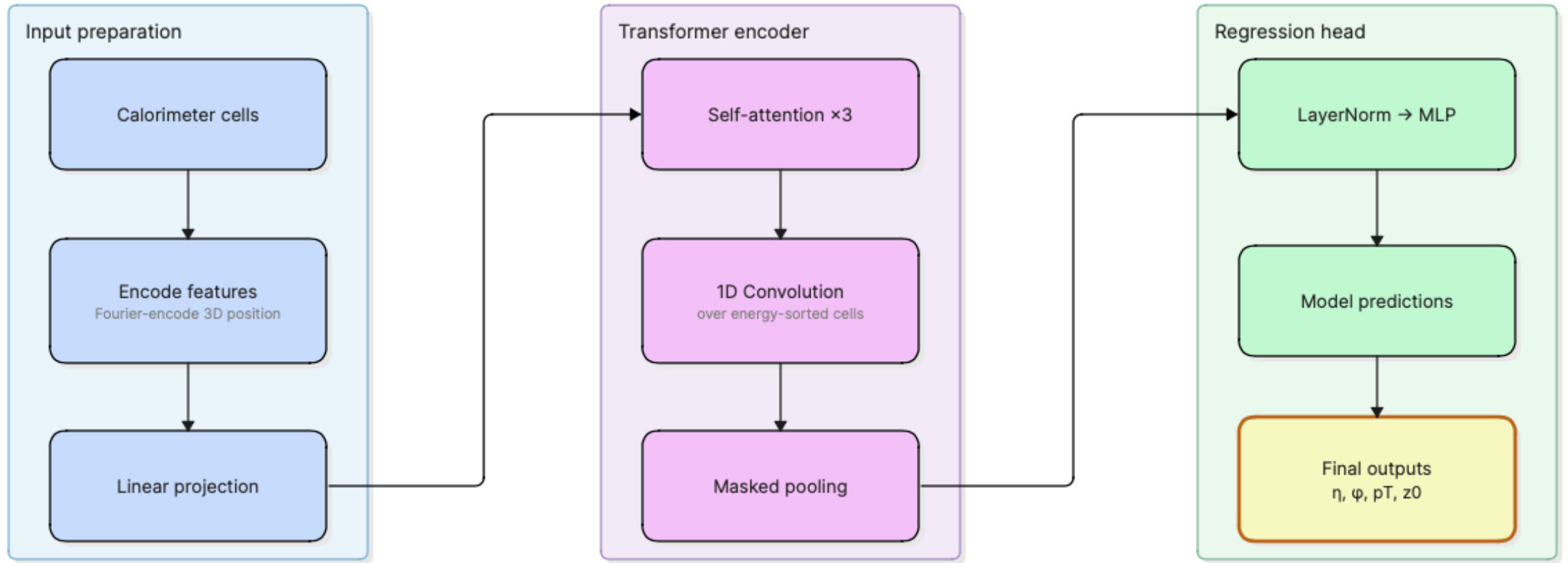
Dataset and model

ColliderML Release 1 dataset

Simulated collisions: Two prompt electrons per event.



Model overview



Model training

Training dataset

- Organized by electron, represented by calorimeter cells in shower
- Outlier cells removed via DBSCAN

Model Input

- Input contains fixed number of highest energy cells per electron
 - Showers with an insufficient number of cells are padded
- Model receives cell data + shower-level features

Input features

Per-cell features

- 3D position
- Angular position
- Cell energy
- Detector

Shower-level features

- Shower energy
- Number of cells in shower
- Angular width: $\sigma_a = \sqrt{\frac{\sum_i E_i a_i^2}{\sum_i E_i}}$
- Angular skew: $\text{skew}_a = \frac{\sum E_i a_i^3}{\sigma_a^3 \sum_i E_i}$
- Radial and longitudinal points of interest

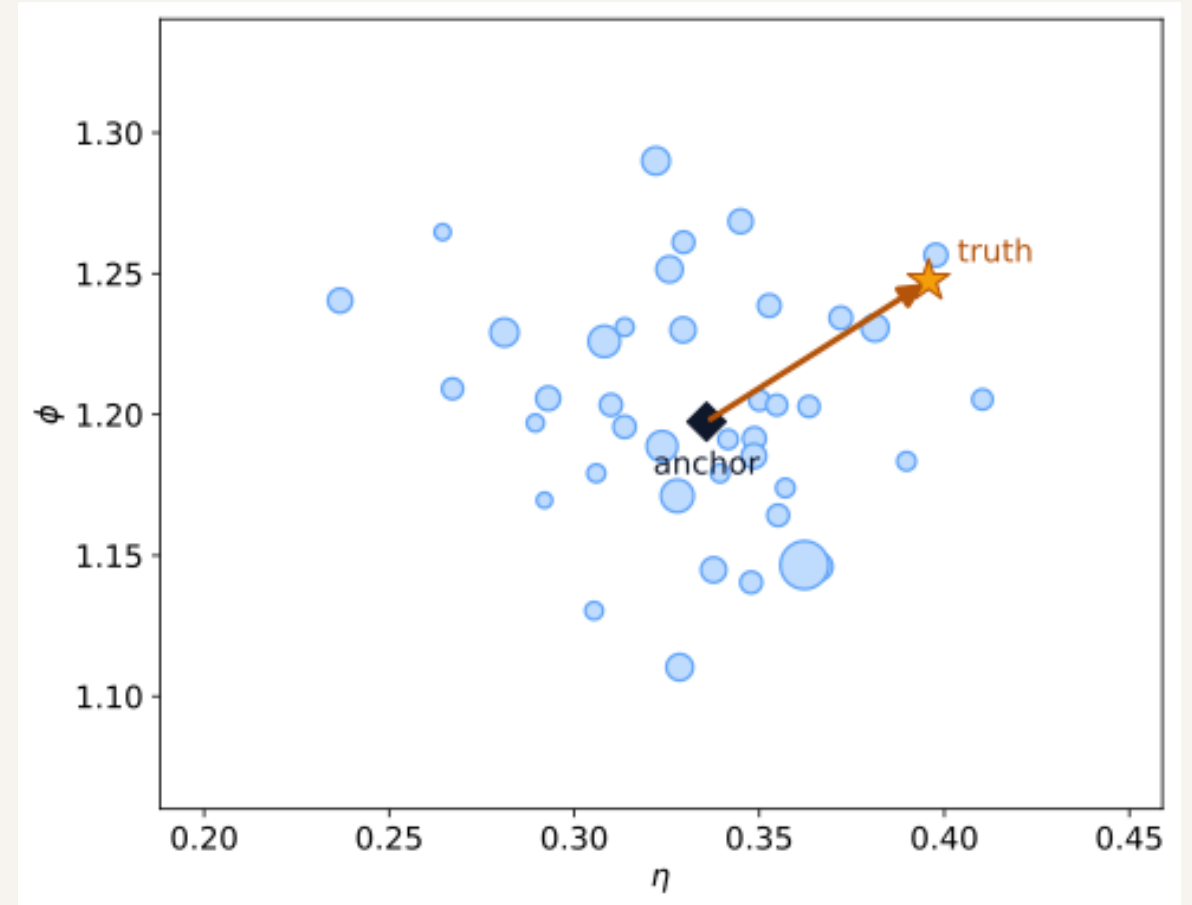
Predicting residuals

Predict offset from physics anchors rather than absolute values.

- $\eta = \eta_c + \Delta\eta$
- $\phi = \phi_c + \Delta\phi$
- $\ln p_T = \ln \sum_i E_T + \Delta \ln p_T$
- $z_0 = \bar{z}_0 + \Delta z_0 b$

Inputs are rotated so $\phi_c = 0$

- Model learns in common frame



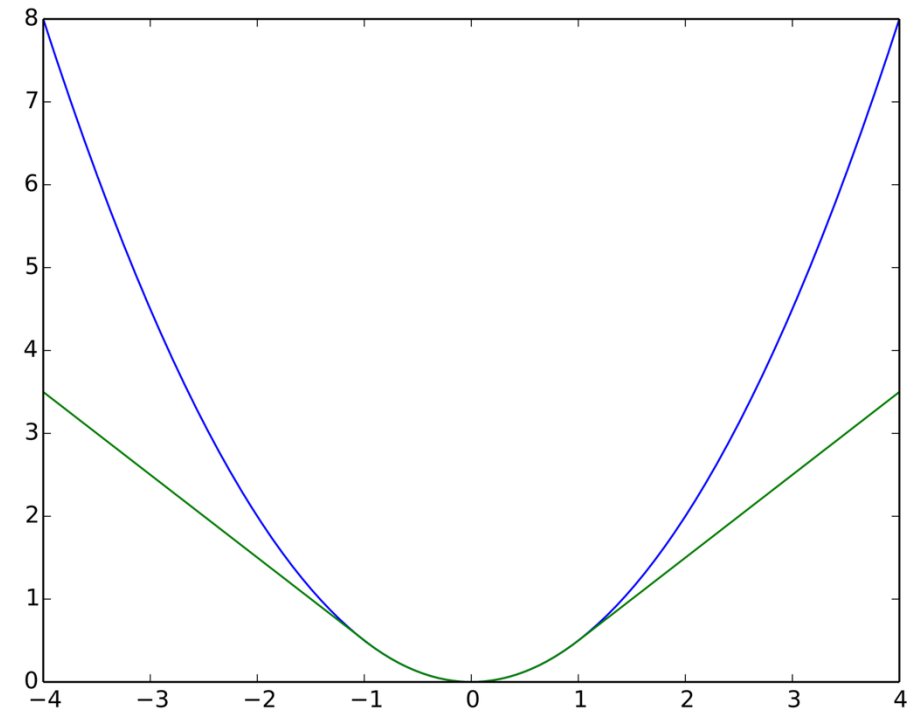
Loss

Huber loss

- Quadratic for small loss, linear for large loss
- Less sensitive to outliers

Homoscedastic uncertainty

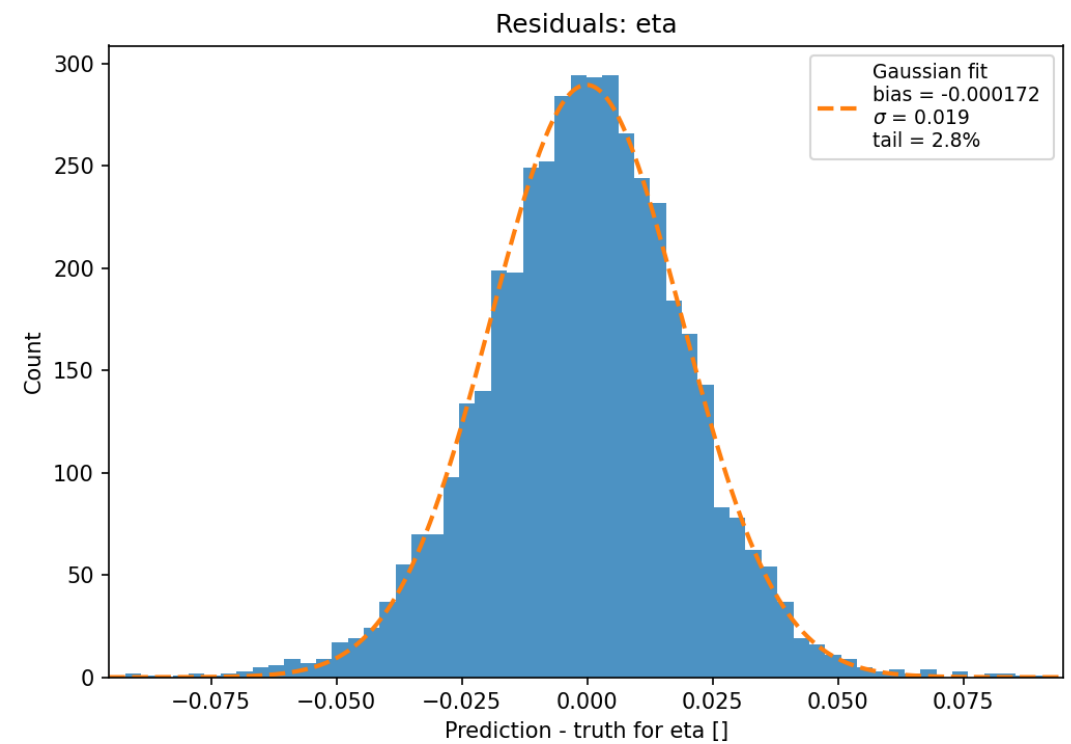
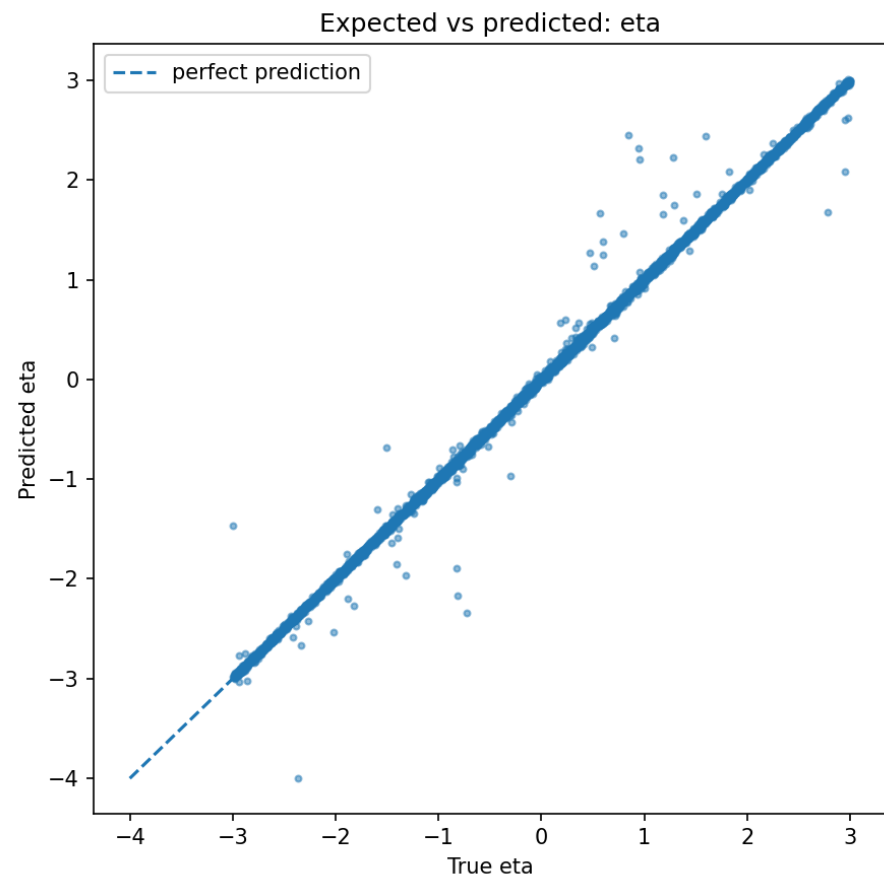
- Individual parameter losses are weighted by uncertainty
- Larger uncertainty $\rightarrow$ less weight



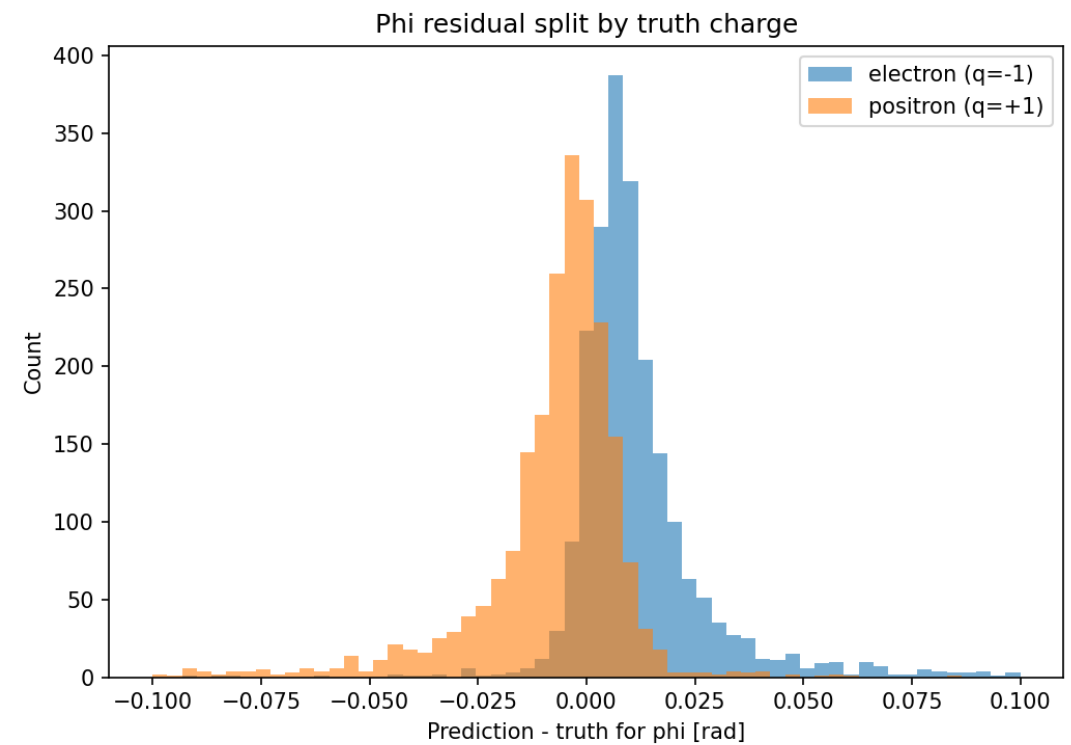
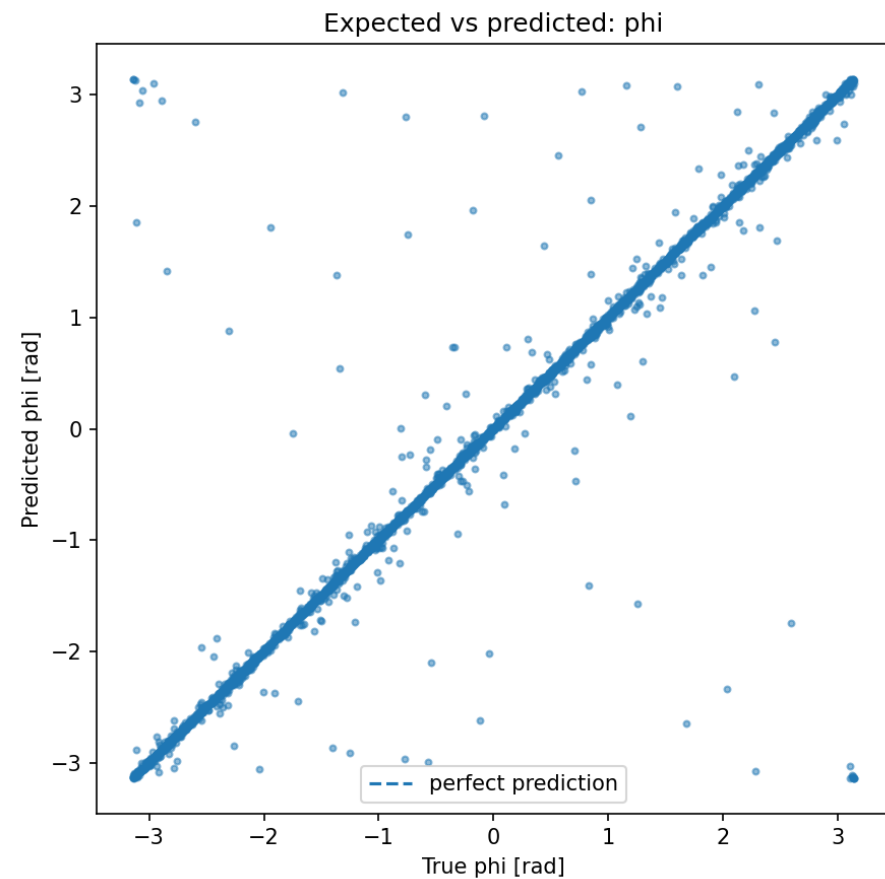
Results

Resolution, bias, and robustness

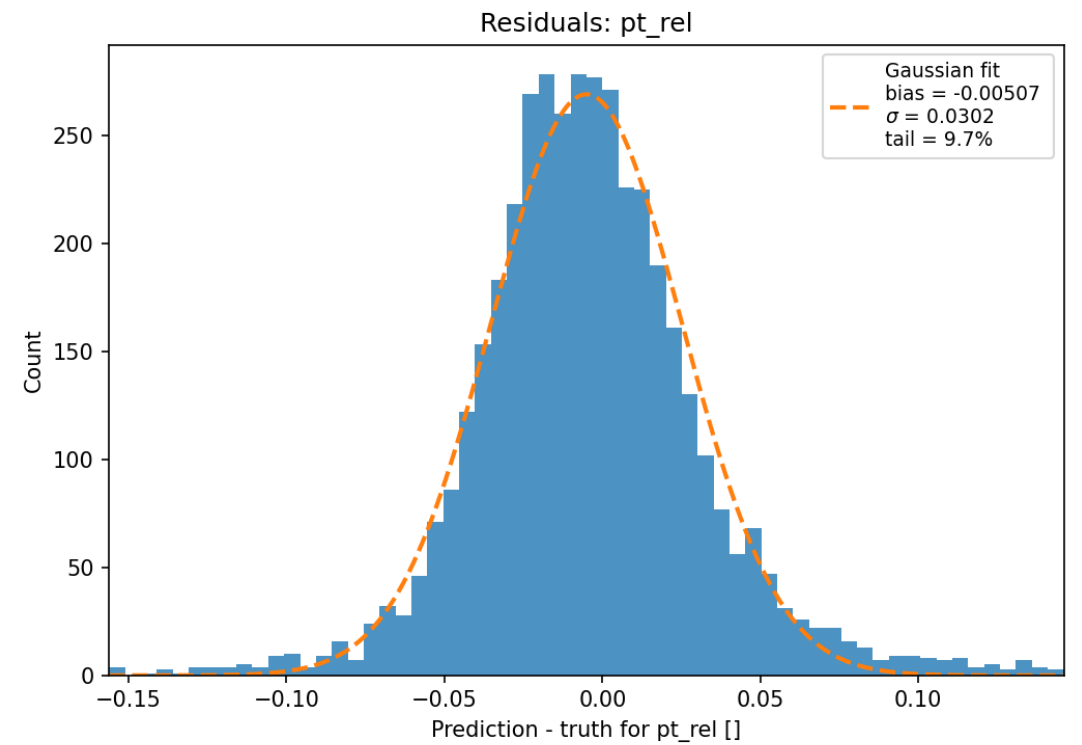
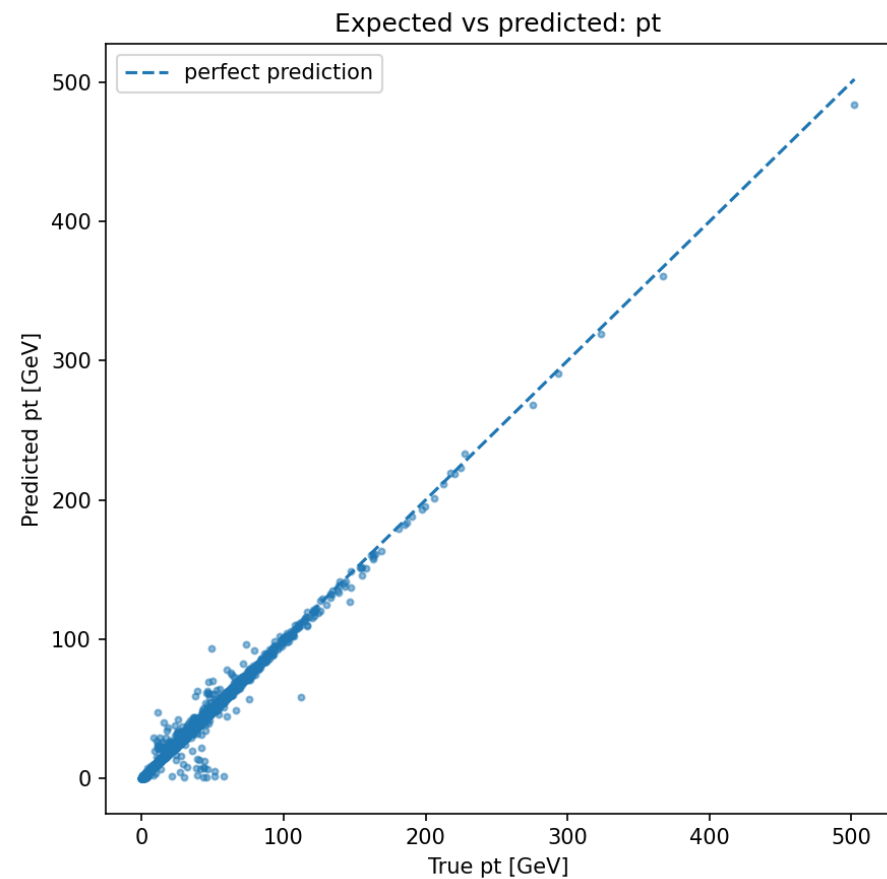
η results



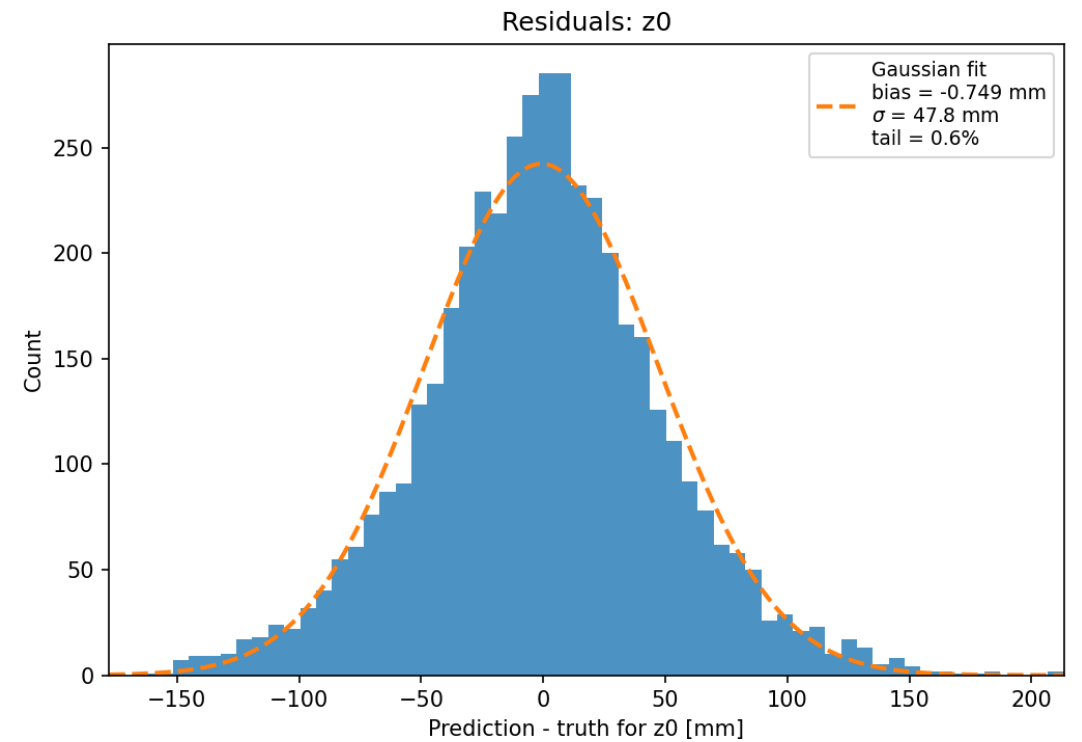
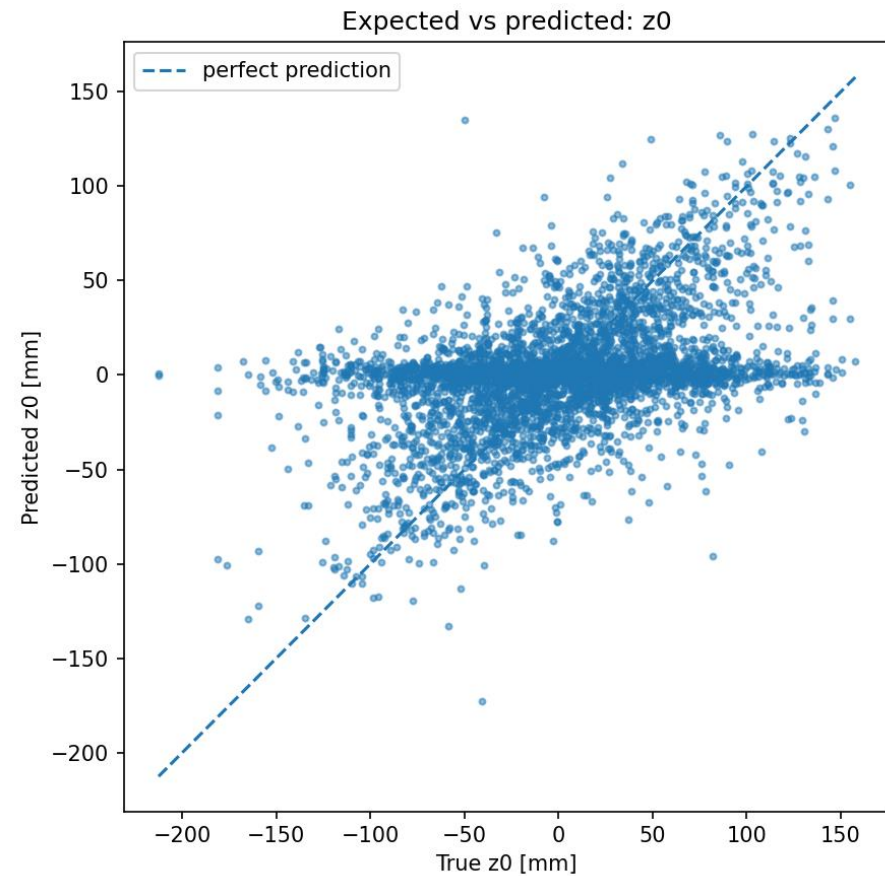
ϕ results



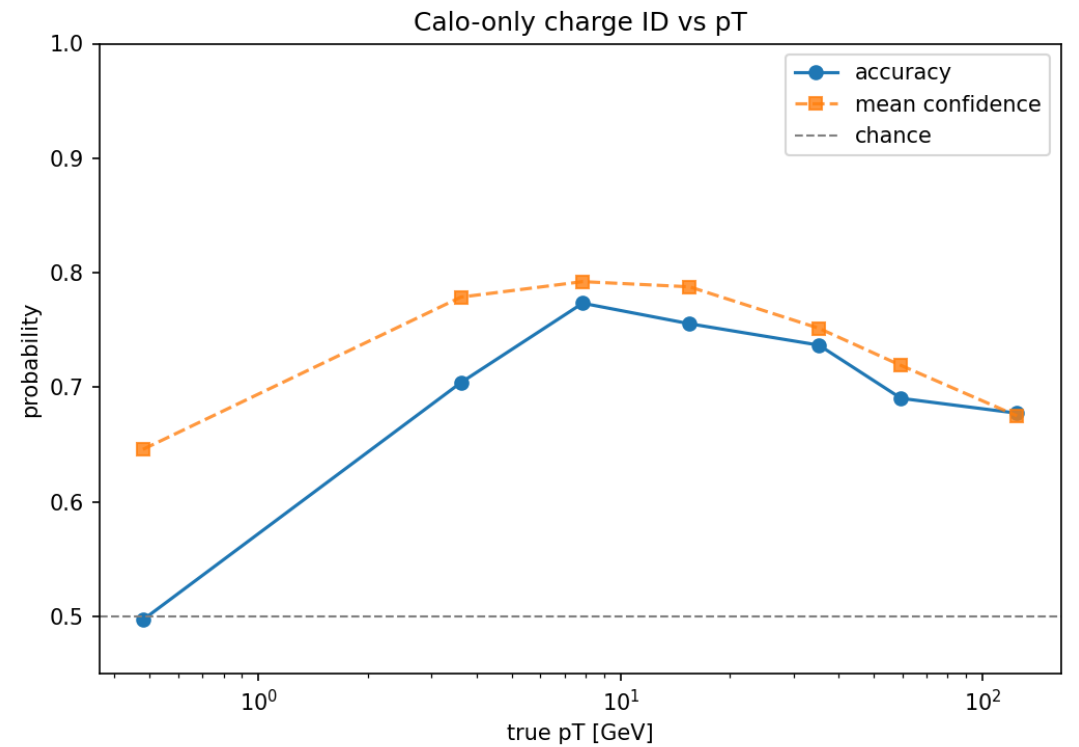
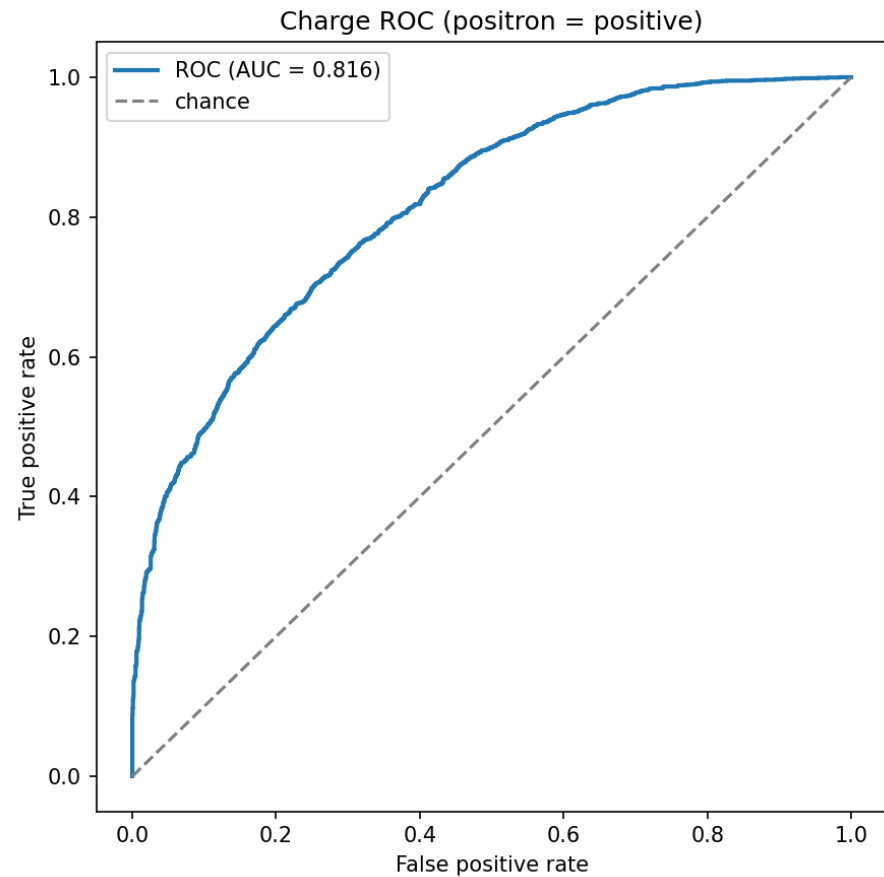
p_T results



z_0 results



q results



Interpretation

Limitations & next steps

Summary

Extra slides

Fourier embedding

Fourier features make it easier for the model to learn spatial patterns

- Each calorimeter cell has a 3D position
 - (x, y, z)
- Instead of using only raw coordinates, coordinates are mapped to sine and cosine features
 - $x \rightarrow \sin(2\pi f x), \cos(2\pi f x)$
- Low-frequency Fourier features capture broad position information.
- High-frequency Fourier features capture fine position information.

Linear projection

Linear projection puts every cell into the model's working dimension

- Each calorimeter cell is represented by an input vector u_i
- The linear projection changes the number of features representing each cell:
 - $u_i \in \mathbb{R}^d \rightarrow h_i \in \mathbb{R}^{128}$
- The linear projection learns combinations of the input features
- A larger model dimension means more complex learning but more computational costs

Self-attention

Self-attention updates each cell by looking at all other cells

- For each cell, 3 vectors are created:
 - $q_i = W_Q h_i$, the query: what the cell is looking for
 - $k_i = W_K h_i$, the key: what the cell contains
 - $v_i = W_V h_i$, the value: the information the cell contributes
- New cell representation $y_i = \sum_{j=1}^L a_{ij} v_j$, where a_{ij} is the attention weight from cell i to cell j .
- The model calculates a_{ij} by comparing cell i 's query to cell j 's key using a dot product.

Convolution

- Calorimeter cells are treated as an energy sorted 1D sequence:
 - $h_1, h_2, \dots, h_L$
- 1D convolution slides a window across the sequence combining cells.
- The same weights W are used at every position, so patterns in the cell sequence can be learned.

For window width k

- $$y_i = b + \sum_{n=-m}^m W_n h_{i+n}, m = \frac{k-1}{2}$$

Masked mean and max pooling

Pooling turns many cell vectors into one shower vector

- The mask indicates which cells are real and which are padded
 - $m_i = 1$ if real calorimeter cell, 0 if padding cell
- For each cell vector entry c :
 - Masked mean pooling calculates $\mu_c = \frac{\sum_i m_i h_{i,c}}{\sum_i m_i}$
 - Masked max pooling calculates $M_c = \max_{i, m_i=1} h_{i,c}$
 - Both vectors $\mu = (\mu_1, \mu_2, \dots, \mu_C)$ and $M = (M_1, M_2, \dots, M_C)$ are concatenated $p = [\mu, M]$ into one final shower vector

LayerNorm + MLP

LayerNorm normalizes components of shower vector $\mathbf{p}$

- $\widetilde{p}_k = \gamma_k \frac{p_k - \bar{p}}{\sqrt{\sigma_p^2 + \epsilon}} + \beta_k$ where γ_k and β_k are learned parameters and $\epsilon \sim 10^5$ to prevent the denominator being 0

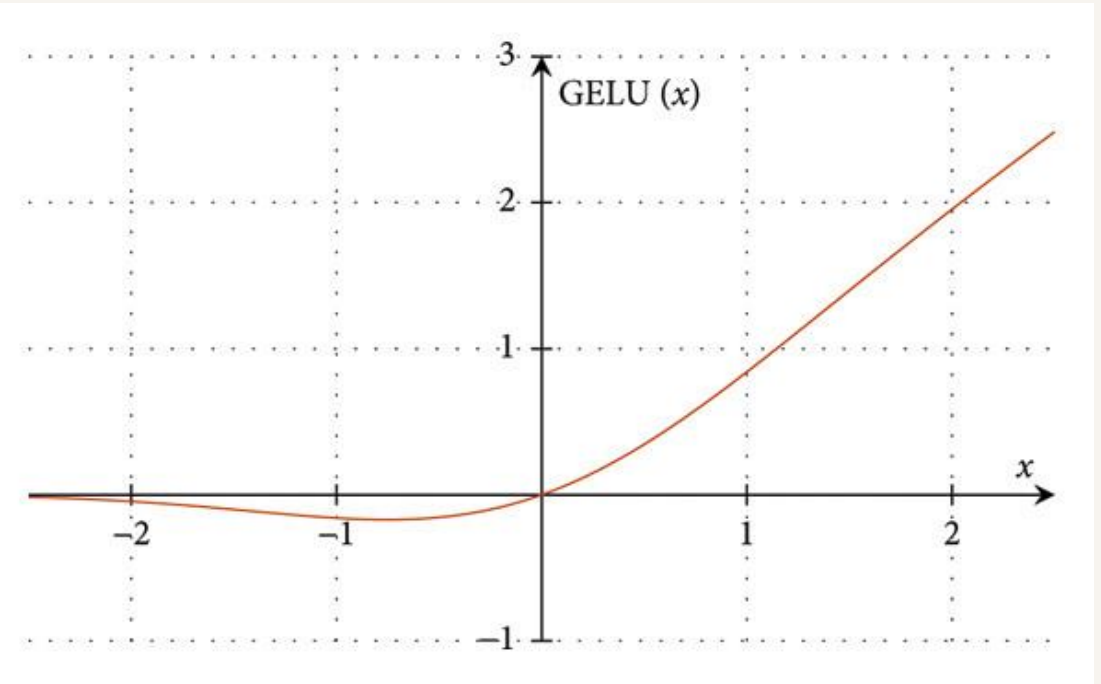
MLP applies learned linear layers and nonlinear activations, yielding predicted electron parameters

- $a_1 = \text{GELU}(W_1 \tilde{\mathbf{p}} + b_1)$
- $\hat{\mathbf{y}} = W_2 a_1 + b_2, \hat{\mathbf{y}} = [\hat{\eta}, \hat{\phi}, \widehat{p_T}, \widehat{z_0}]$

GELU

GELU lets the MLP learn nonlinear relationships

- $\text{GELU}(x) = x\Phi(x)$, where $\Phi(x)$ is the cumulative distribution function of a standard normal Gaussian
- GELU keeps strong signals and smoothly suppresses weak ones



Learned parameters

A learned parameter θ is adjusted by gradient descent training

- $\theta = \theta_0 - \alpha \frac{\partial \mathcal{L}}{\partial \theta}$, where $\mathcal{L}$ is the loss and α is the learning rate
- Each learned parameter has the same loss function but a different gradient.
- The loss is calculated between the predicted and true electron parameters.

Homoscedastic uncertainty weighting

A loss weight for each parameter is learned via an uncertainty parameter

- $\mathcal{L}_{\text{tot}} = \sum_i \frac{\mathcal{L}_i}{2\sigma_i^2} + \ln \sigma_i$
- Larger σ means lower weight
- Lower σ means larger weight
- $\ln \sigma$ term to prevent the model from cheating by setting $\sigma = \infty$
- σ is a learned parameter